Assignment 5: Mobile Manipulation Project

- Due Oct 11, 2024 by 5pm
- Points 0
- Submitting a file upload
- File Types py and launch
- Available Sep 13, 2024 at 12am - Dec 31, 2024 at 5pm

This assignment was locked Dec 31, 2024 at 5pm.

Both students in the pair must upload their solutions (sm_students.py, bt_students.py, launch_project.launch) here. Please write a comment regarding who you have worked with. This is important in order to avoid false plagiarism reports.

Assignment 5: Mobile Manipulation Project

This assignment is carried out in pairs, and presented orally to the TA's. You must upload your files here before the oral presentation.
Completing this assignment to an E level by Oct. 11, 17:00 will give you 3 bonus points towards the score of the final exam. Completing it to the C level by the same deadline will give you 5 bonus points, and the A level will give you 7 points.

Assignment presentation

Presentation slides (https://canvas.kth.se/courses/48906/files/7969160?wrap=1) ↓ (https://canvas.kth.se/courses/48906/files/7969160/download_frd=1)

Introduction

The final project consists of a simulation in **Gazebo (http://gazebosim.org/)** of a **TIAGo robot (http://tiago.pal-robotics.com/)** in an apartment. The robot is equipped with several onboard sensors and a manipulator arm which enable it to autonomously navigate and interact with the environment through simple manipulation tasks. The goal of this project is to implement a mission planner for TIAGo to execute three different missions. **Note that you need to upload your solution here in Canvas before presenting, details below.**

Format

In a real robotics team, you usually collaborate with other developers who are often responsible uniquely for a single module of a large robot system. Usually things work well until it's time to put them together and prepare the robot for a specific setup... This is the scenario of this project.

You are provided with a real set of working modules (planning, sensing, navigation, manipulation...) and you have to design and integrate a mission planner node with the necessary logic for the robot to carry out a given mission making use of these modules. This node will be responsible for the high-level task planning of the system, commanding where to go and what to do based on the robot current state and the mission specifications.

For this, it will have to utilize the available sensors and actuators through the software packages provided. Unfortunately, just like in many real system integration projects, most of these packages come with very little documentation and your colleagues are not around anymore, so you will have to do some digging.

"It reminded me of my first day working at Boeing, and it wasn't a good day".

- Former student

You are expected to implement and show a working solution consisting of two parts:

1. The mission planner node, within the corresponding files:
    1. "sm_students.py" in which you have to implement a high-level state machine (SM) triggering the right sequence of steps for the robot to achieve the final goal. In order to come up with this sequence, look at the videos provided and go through all the topics, services and actions offered by the modules provided. If you need the robot to do a backflip, most likely there's a

service called /backflip_jump or similar. An example of state machine is provided within the working package.

2. "bt_students.py" contains a **behavior tree** (http://wiki.ros.org/py_trees_ros) (BT) equivalent to the state machine above (for grade C or A).

2. A **launch file** (http://wiki.ros.org/roslaunch/XML) "launch_project.launch" where you have to deploy and connect the nodes/components of the system to carry out the tasks. For each different task you will have to uncomment the nodes and provide them with the parameters required (substitute place_holderX with meaningful names).

Use the same node in the launch file to launch either the SM or BT just by changing the name of the python script being called, depending on the task you are solving

```
<node pkg="robotics_project" type="sm_students.py" name="logic_state_machine" output="screen">
```

The files mentioned are under the **robotics_project** package, available for you in the repository you are about to clone. You do not have to modify any other file than these three.

There are three different missions with increasing difficulty levels, which will give you a corresponding grade. The reasonable way to go is to start with E and move on from there, since the higher levels build conceptually on top of the previous ones (with some changes in the implementation), but it's ok if you don't want to be reasonable.

Along with the demos, you will be questioned about basic concepts of the solutions you have implemented (see examples below). **The tasks are considered solved if your system works and you are able to answer these questions during the presentation**.

Task E: Pick&Carry&Place without sensory input

In this scenario, TIAGo is expected to pick an Aruco cube from a known pose on top of table 1, navigate towards table 2 behind it and place the object there. The cameras and laser scan cannot be used for this level.

Implement a **state machine** which goes through the following main states:

1. Complete picking task
2. Carry cube to second table
3. Complete placing task

Obs: use the ROS tools (rostopic, rosmsg, rosservice, rosrun tf view_frames, etc) to explore the system and figure out which module does what. You will realize this is the most time-consuming part of this level. Once you are familiar with the project, implementing the solution will be straight forward.

Evaluation:

1. Show a working simulation and be able to explain your implementation
2. Be able to reason about this solution, i.e: Can the mission succeed if the cube is displaced before being picked? What if table 2 is moved?

Task C: Pick&Carry&Place with visual sensing

The task is the same as above. However this time the camera sensor in TIAGo's head has to be used to detect the cube. After, compute a grasp, transport the marker and verify that it has been placed on the second table. You will implement this logic in the form of a behavior tree this time.

Implement a **behavior tree** which goes through the following main states:

1. Detect cube
2. Complete picking task
3. Carry cube to second table
4. Complete placing task
5. Cube placed on table?
    1. Yes: end of task
    2. No: go back to initial state in front of table 1

Evaluation:

1. Show a working simulation and be able to explain your implementation
2. Be able to reason about this solution, i.e: Can the mission succeed if the cube is displaced before being picked? What if table 2 is moved? Would the robot be able to transport several cubes (one at a time) with the given solution?

Task A: Pick&Carry&Place with sensing and navigation

Pick&Carry&Place with visual sensing and **navigation (http://wiki.ros.org/navigation?distro=kinetic)** : in this third level, the robot **starts in an unknown pose (https://en.wikipedia.org/wiki/Kidnapped_robot_problem)** and must make use of its sensors and a prior map of the room to transport the cube safely among rooms.

Implement a **behavior tree** that goes through the following main states:

1. Robot has localized itself in the apartment
2. Navigation to picking pose
3. Cube detected
4. Complete picking task
5. Navigation with cube to second table
6. Complete placing task
7. Cube placed on table?
    1. Yes: end of mission
    2. No: go back to state 2 and repeat until success. For this, you need to respawn the cube to its original pose in case it has fallen.

Obs 1: At any time during the navigation (not after the picking/placing sequences have started), a bad-intentioned TA might kidnap your robot. Your behavior tree must be able to detect this and react to it so that the robot always knows its true position and can avoid collisions. Kidnap the robot yourself during your implementation to test your solution (the simulation can be paused and the robot moved manually, see how **here** (http://gazebosim.org/tutorials?tut=guided_b2&cat=)).

Obs 2: The TA might also manually throw away the cube from the robot's gripper during the navigation to see that the robot is able to detect that the placing has failed. As before, you can test this yourselves by dropping the cube through the Gazebo GUI.

Obs 3: The robot uses a particle filter for localization. Use the current state of the distribution of the particles to know when the filter has converged. Other solutions will not be accepted.

Obs 4: You may not use the true state of the models, i.e., reading `/gazebo/model_states` to know if you failed to grasp the cube, or to know if you have lost localization.

Evaluation:

1. Show a working simulation and be able to explain your implementation
2. Be able to reason about this solution, i.e: Why do we ask you to make the robot spin to help the AMCL? What does the distribution of particles tell us? Why does AMCL fail to converge sometimes? When to use/avoid timers while waiting for a result from an action server.

Install

The following instructions are for the PCs in the lab rooms, which have already been set up for you.

```
# Download the project code:
# From files, download assignment_5.zip (https://canvas.kth.se/courses/48906/files/7969139?wrap=1) ↓
 (https://canvas.kth.se/courses/48906/files/7969139/download?download_frd=1)

# Unpack it in ~/catkin_ws/src/
# (or using the file browser)
cd ~/catkin_ws/src
unzip ~/Downloads/assignment_5.zip

# Add this line at the end of your .bashrc file and source it:
export GAZEBO_MODEL_DATABASE_URI=http://models.gazebosim.org/
source ~/.bashrc

# Build the project:
cd ~/catkin_ws
```

```
catkin_make -DPYTHON_EXECUTABLE=/usr/bin/python3
```

```
source devel/setup.bash
```

To run the project in personal computers, your own missing system dependencies have to be met. A hint on how to solve this can be found in this **README** (https://github.com/ignaciotb/robi_final_project/blob/master/README.md) but the setups will vary for each of your installations and we will not offer support for this (Google is your friend here). As an extra advice, stay away from virtual machines and make sure your laptop can handle this workload.

Launch the simulation

To run the project, execute the following commands in two terminals:

```
# Launch Gazebo and RViZ
roslaunch robotics_project gazebo_project.launch

# Deploy the system and start the simulation
roslaunch robotics_project launch_project.launch
```

Wait for Gazebo and RViZ to show up before running the second command.

While developing and testing, you can stop (Ctrl+C) the launch_project.launch script and relaunch it without having to relaunch the gazebo part. This will make things faster and reduce the risk of issues related to relaunching the Gazebo scenario several times. If you do not want to run the Gazebo graphical interface (client) in order to reduce the workload of your PC, you can set the "gzclient" variable in the launch file to false.

If everything works out, you should see the robot moving around the apartment, folding its arm, approaching a chair and lowering its head. This dummy example (also available as a BT) shows you how to call services and actions and interact with topics from your mission planner. They are good skeletons to start to develop your own solutions.

Errors out of the scope of your solution

You might experience that the simulation fails sometimes despite the fact that all components are correctly implemented (much like in a real system). **MoveIt!** (https://moveit.ros.org/) may fail to

compute a trajectory for the robot arm while picking/placing (i.e Pick result: PLANNING_FAILED) or the **Navigation stack (http://wiki.ros.org/navigation)** might get the robot stuck in a spinning loop while trying to reach a waypoint due to very small inaccuracies on the base localization.

Handling these errors is not requested beyond logging them and showing them on terminal from your mission planner. If they occur during the demo, you will be given another try to run the full simulation, so do not despair!

A full, successful round of the level A solution might take a while and during the testing it can be frustrating to see the robot fail for the reasons above. A couple of hints in order to save time **during the development** of your solutions:

Hint 1: Sometimes the robot might collide with the leg of the picking table while navigating towards the second table. It's ok to pause the simulation and manually give a little push to the robot away from the table if it gets stuck (but put the table back after).

Hint 2: Equivalently, if the robot cannot reach the pick waypoint and instead starts to spin in front of the table (as explained above), manually place it a bit further and let it recompute the path and try again.

Hint 3: If you don't want to wait for the robot to slowly navigate from table A to table B after picking the cube, kidnap it yourself and place it next to the waypoint!

In general, you're allowed to modify and play with the full simulation scenario you've been given in order to make your life easier during your development. Everything is accepted **as far as your solution works as expected during the examination in the original scenario**.

Submit your files

For this assignment, **you have to upload your solution files here in Canvas BEFORE YOU PRESENT YOUR SOLUTION**. You should upload your files "sm_students.py", "bt_students.py" and "launch_project.launch", as required for the different grades. Use the "Submit Assignment" button at the top right of this page.

Work in pairs

In this assignment, you will have to work in pairs. Try to pair up with another student with the same level of ambition. You can use the "Discussions" forum to look for partners. You must have a compelling reason to work on your own. Let us know as soon as possible.